AuraZone website Testing

1. TEST PLAN

1.1 Introduction

The purpose of this Test Plan is to define the testing scope, approach, resources, timelines, and exit criteria for evaluating the web application **AuraZone Test Store** (https://test.aurazone.shop/) as part of the QA Internship technical assessment.

Testing focuses on validating functional behavior, UI/UX, form validation, cart & checkout workflow, responsiveness, and basic automation stability.

1.2 Scope

In-Scope

Area	Covered in Testing
Product Listing Page	Rendering, inventory status, add to cart
Product Detail Page	Navigation, UI elements
Shopping Cart	Quantity updates, price calculations, persistence
Wishlist	Move-to-cart logic, stock indicators
Checkout Page	Form validation, payment selection, error handling
Search Function	Valid & invalid queries
Responsiveness	Mobile browser views
Automation	Five critical flows automated via Selenium

Out-of-Scope

- API testing
- Security testing (SQL injection, XSS, rate limiting)
- Backend validation of orders
- Integration with payment gateways

1.3 Features to be Tested

1. Product Catalog Page
2. Product Detail Navigation
3. Add to Cart / Remove from Cart
4. Wishlist behavior
5. Checkout form validation
6. Order placement flow
7. Search functionality
8. Mobile responsiveness
9. Toast notifications behavior
10. Inventory consistency (In Stock vs Out of Stock)

1.4 Types of Testing

Testing Type	Description
Functional Testing	Validate all user workflows end-to-end
UI/UX Testing	Layout, alignment, notifications, responsiveness
Validation Testing	Form fields, input restrictions, mandatory fields
Cross-browser Testing	Chrome, Firefox, Edge
Mobile Testing	iPhone 12, iPhone SE, Samsung S8 views
Data Integrity Testing	Stock status & price accuracy
Automation Testing	Five automated test cases (checkout validations)
Exploratory Testing	Free-flow bug discovery

1.5 Assumptions

- The test environment is stable.
- No rate-limiting or IP blocking occurs if requests are kept minimal.
- Out-of-Stock logic is controlled via backend flags and not mock data.
- Email and tracking URLs are functional for test orders.

1.6 Exit Criteria

Testing is considered complete when:

- All planned test cases are executed.
- Critical & major bugs are documented.
- Automation scripts run end-to-end without breaking.
- Cross-browser tests show consistent behavior.
- Final documentation (Test Plan, Bug Report, Insights) is delivered.

2.EXPLORATORY TESTING

All testing along with test cases

TC-09	Payment Selection	Verify selection of Payment Method	User is at Checkout.	1	1. Click "Cash on Delivery."	The radio button selection should change correctly.	Radio Button selection changing Correctly	Passed
				2	2. Click Online Payment			
TC-10	Mobile Responsive	Verify "Shop Now" button visibility on mobile.	User is using Chrome DevTools (Mobile View)	1	1. Set view to iPhone 12.	"Shop Now" button should be centered and clickable.	Shop Now button is visible	Passed
				2	2. Observe the main Home banner.			
TC-11	UI/UX - Layout	Verify aspect ratio consistency for product images.	User is on the "Most Popular" or Catalog page.	1	1. Observe the product cards for "Retro Colorblock" vs "Vibrant Pulse.	All product images should have a consistent aspect ratio (e.g., all square) for a professional look	Images have inconsistent orientations (some horizontal, some vertical), leading to an uneven grid	Failed
TC-12	Product Data	Verify price visibility for "Out of Stock" items.	A product is marked as "Out of Stock."	1	1. Locate the "Azure Elegance" product card.	The price should remain visible so customers can decide to wait for a restock and there should be notify me button	The price is hidden and replaced by the "OUT OF STOCK" button	Failed
				2	2. Look for the product price.			
TC-13	Order Tracking	Verify that "Track Order" button in confirmation email authenticates the user.	User has placed an order and received a confirmation email.	1	1. Click the "Track My Order" button in confirmation email	The tracking page should open with the unique token already applied, showing live order status without requiring manual login.	The page opened, the token was automatically passed in the background, and live tracking was visible.	Passed
				2	2. Observe the Tracking Page URL and content.			
TC-14	Form Validation	Verify name field rejects numeric-only input.	User is on the Checkout page.	1	1. Enter "555555555555555" in Name feild .	The field should require alphabetic characters and show a validation error for numeric strings.	The system accepted the numeric string and generated an Order Confirmation addressed to "Hello 555555555555555"	Failed
				2	3. Complete checkout.			
TC-15	Form Validation	Check PIN Code field for non-numeric input.	User is on the Delivery Address section.	1	1. Locate the "PIN Code" field.	System should not allow user to enter	system is not allowing alphabetical input	Passed
				2	2. Enter letters "ABCDE"			
				3	3. Click "Place Order.			
TC-16	Wishlist Logic	Verify "In Stock" status for Out of Stock items in Wishlist.	"Azure Elegance" is confirmed as Out of Stock on the product page.	1	1. Add "Azure Elegance" to Wishlist.	The item should be marked as "Out of Stock" to match the actual inventory.	The Wishlist incorrectly displays a green "In Stock" label and shows the price.	Failed
				2	2. Open the "My Wishlist" page.			
TC-17	Form Validation Automation	Verify mandatory fields block submission when empty.	User is on the Checkout page.	1	Click "Place Order" with all fields empty.	System should display validation errors for required fields.	Form remained on page; validation triggers appeared	Passed

Test Case

Test Case ID	Feature	Description	Pre-conditions	Serial Number	Test Case	Expected Result	Actual Result	Status
TC-01	Inventory	Check if "Out of Stock" items can be added to the cart.	"Azure Elegance" is marked as Out of Stock.	1	1. Open Azure Elegance page	Buttons should be disabled (unclickable) and show "Out of Stock."	Buttons were disabled and shows Out of Stock	Passed
				2	2. Click "Add to Cart" or "Buy Now" buttons.			
TC-02	UI/UX	Verify toast notification behavior on multiple updates.	User has an item in the cart.	1	1. Go to Shopping Cart.	Notifications should appear in a non-intrusive way (e.g., a single popup or a clean queue).	Multiple "Quantity updated" boxes stack vertically, obscuring the header and creating a cluttered UI.	Failed
				2	2. Click the "+" button rapidly 5 times.			
TC-03	Error Handling	Verify clarity of error messages when moving items to Cart.	Item in Wishlist is actually Out of Stock.	1	1. On Wishlist page, click "Move to Cart" for "Azure Elegance".	Error message should be specific: "Cannot move to cart: Item is out of stock".	System shows a vague "Failed to move item to cart" message, providing no reason to the user.	Failed
				2	2. Observe the error notification			
TC-04	Calculations	Verify Subtotal + Shipping calculation	User has items in the cart.	1	1. Note Subtotal and Shipping	The "Total" must exactly match Subtotal + Shipping.	Total matches (₹475 + ₹200 = ₹675).	Passed
				2	2. Add them together.			
TC-05	Search	Verify search for invalid products.	User is on Home page	1	1-Search for "Pizza "	Display: "No products found".	No Product Found	Passed
				2	2- observe Result			
TC-06	Navigation	Verify "Back" button functionality on product pages	User is on a Product Detail Page.	1	1. Click the "<- Back" button at the top left	User should be redirected to the previous page (Home/Catalog).	User Redirected to Previous page	Passed
TC-07	Cart Persistence	Verify cart items remain after page refresh.	User has items in the cart.	1	1. Add item to cart.	Cart should still contain the added items.	items remian after page refreshed	Passed
				2	2. Refresh the browser (F5)			
				3	3. Check the cart icon.			
TC-08	Form Validation	Check mandatory field validation for Street Address.	User is at Checkout.	1	1. Leave "Street Address Line 1" empty.	System should prevent order and highlight the missing field.	System pop up an error for missing feild	Passed
				2	2. Fill all other fields.			
				3	3. Click "Place Order."			

TC-18	Data Integrity Automation	Verify the Name field does not accept numeric characters.	User is on the Checkout page.	1	Enter 555555555 into the Full Name field.	System should show an error: "Name cannot contain numbers."	Field accepted numeric input without error.	Failed
TC-19	Email Validation Automation	Verify the system detects invalid email formats.	User is on the Checkout page.	1	Enter areesha.com into the Email field.	System should show an error: "Invalid email format."	Red error message "Invalid email" appeared.	Passed
TC-20	Data Logic Automation	Verify the PIN Code field enforces a standard length.	User is on the Checkout page.	1	Enter 123 into the PIN code field.	System should require a minimum of 6 digits.	Field accepted a 3-digit PIN.	Failed
TC-21	Selection Logic Automation	Verify payment method selection updates correctly.	User has filled contact details.	1	Click the "Cash on Delivery" radio/button	"Cash on Delivery" should be highlighted/selected.	Button selected successfully as confirmed by script.	Passed

2.BUG REPORTING

Bug 01 — Wishlist Shows WRONG Stock Status

Severity: High
Module: Wishlist

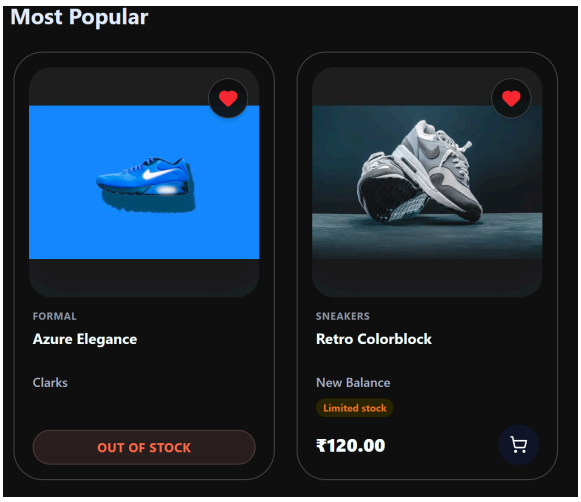
Description:
Out-of-stock items appear as **"In Stock"** after being added to the wishlist.

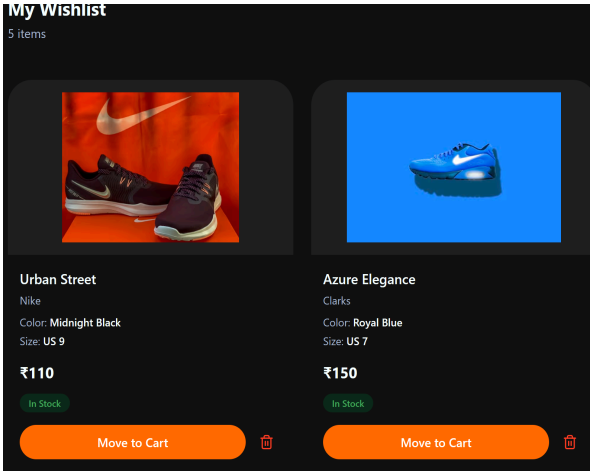
Steps to Reproduce:

1. Open "Azure Elegance" (marked Out of Stock).
2. Add to Wishlist.
3. Open Wishlist page.

Expected Result:
Status should remain **Out of Stock**.

Actual Result:
Item shows **In Stock**, misleading the user.





Bug 02 — Numeric Customer Name Accepted in Checkout

Severity: High
Module: Checkout Form

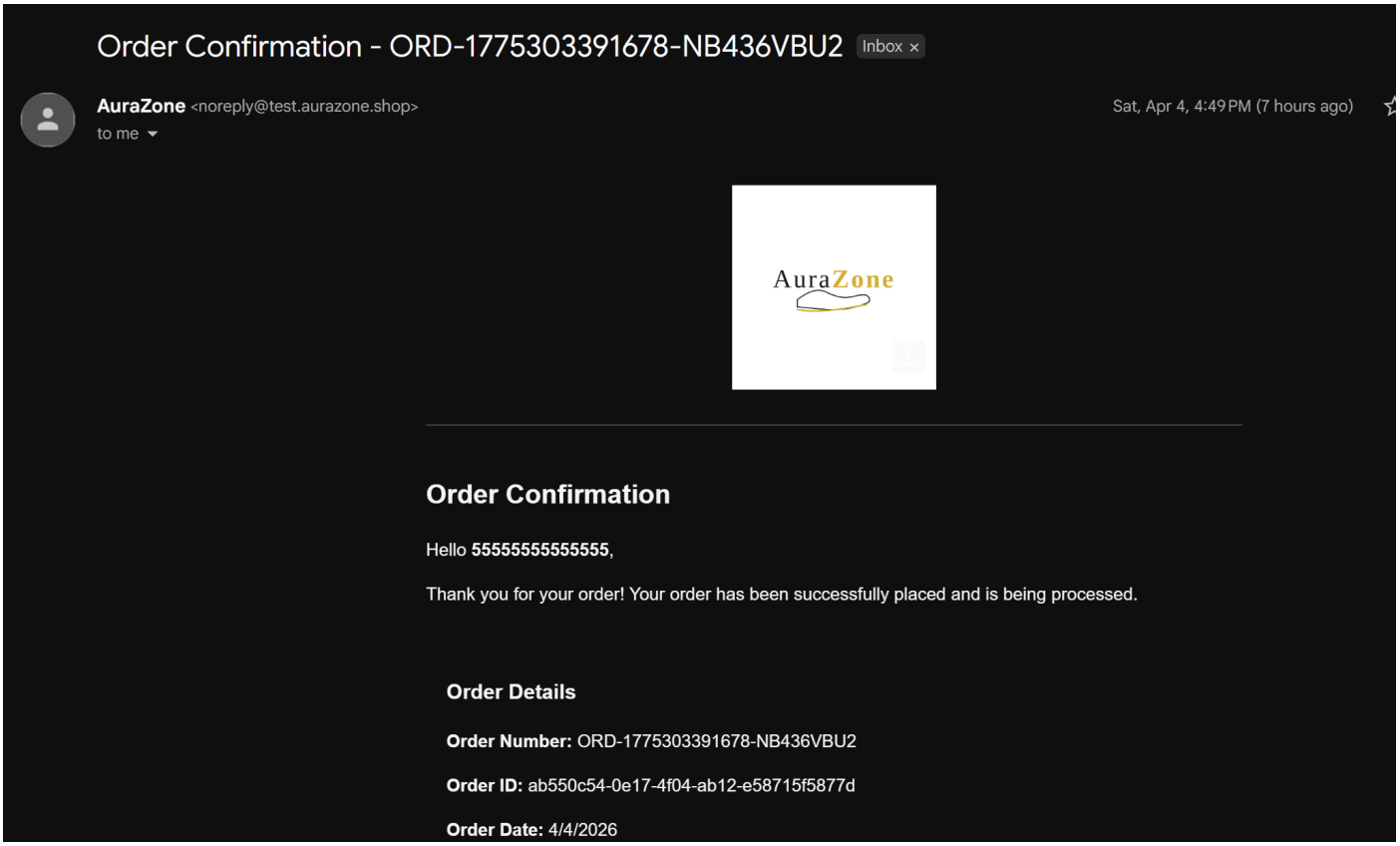
Description:
The system accepts numeric-only names.

Steps to reproduce:

1. Open Checkout.
2. Enter Name: **"555555"**
3. Submit Order along with other details .

Expected:
Validation error ("Name cannot contain numbers").

Actual:
Order is accepted without any validation error.



Bug 03 — Short PIN Code (Less Than 6 Digits) Accepted

Severity: Medium
Module: Shipping Details

Steps to reproduce:

1. Enter PIN code: **“123”**
2. Submit order.

Expected:
Minimum 6-digit PIN required.

Actual:
Order proceeds with invalid PIN.

Screenshot shows that 087 is PIn code

Bug 04 — Toast Notifications Overlap & Stack Improperly

Severity: Medium
Module: Cart

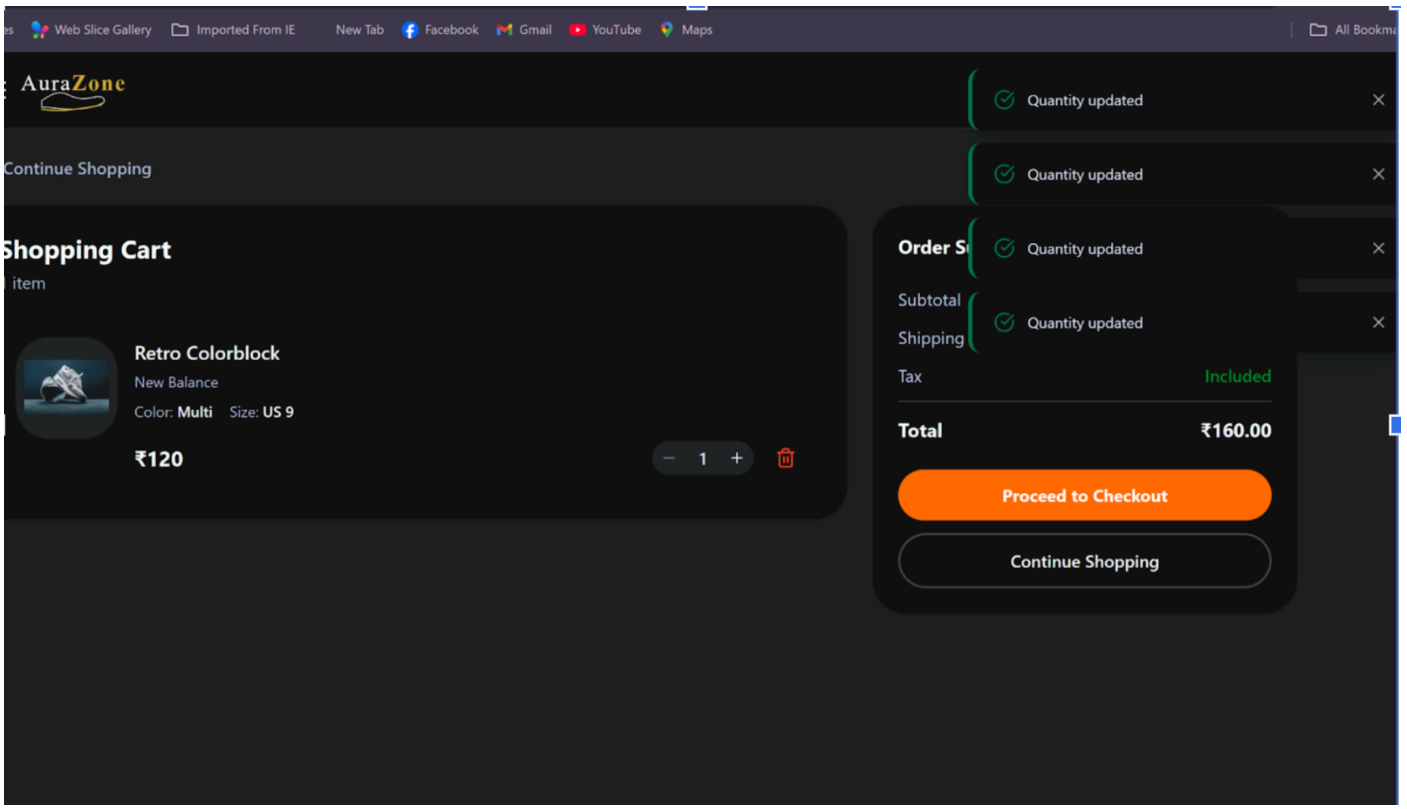
Description:
Multiple toast notifications stack on top of each other and cover the UI.

Steps to reproduce:

1. Open Cart page.
2. Quickly click “+” or “-” quantity button 4 times

Expected:
Single toast or queued messages.

Actual:
4 overlapping toasts blocking header area.



Bug 05 — Out-of-Stock Product Price Hidden Completely

Severity: Medium
Module: Product Listing UI

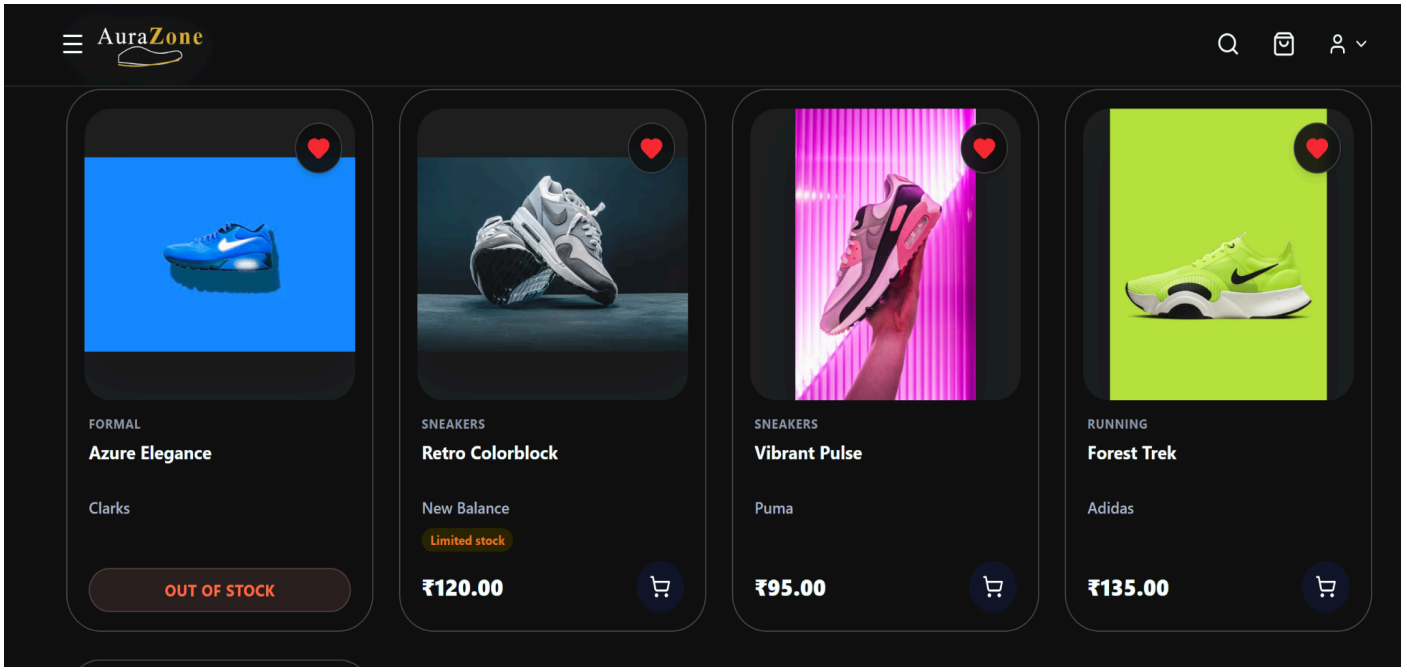
Description:
For OOS items, the price disappears, causing user confusion.

Steps to reproduce :

1. Locate “Azure Elegance” on product listing.
2. Observe the price label.

Expected:
Price should remain visible but greyed-out or struck-through.

Actual:
Price not shown at all.



Bug 06 — Error Message Not Informative When Moving Out-of-Stock Item to Cart

Severity: Medium–High
Module: Wishlist → Move to Cart

Description:

When attempting to move an Out-of-Stock item ("Azure Elegance") from the Wishlist to the Cart, the system displays a vague error message. It does not tell the user *why* the action failed.

Steps to Reproduce:

1. Add the product **"Azure Elegance"** (status: *Out of Stock*) to the Wishlist.
2. Open the **Wishlist** page.
3. Click **"Move to Cart"** on this product.
4. Observe the error message.

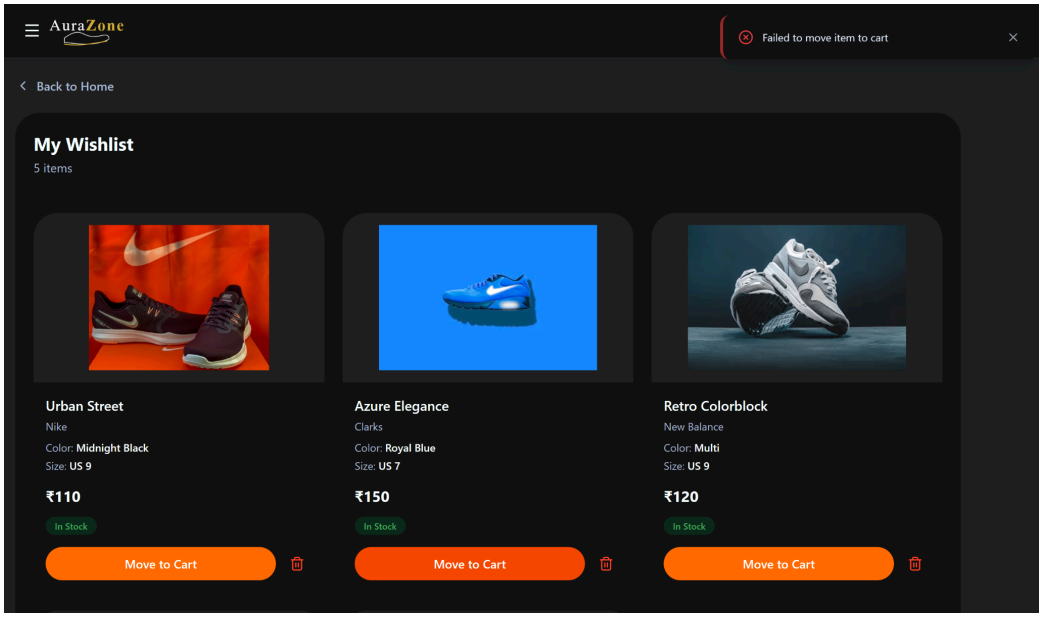
Expected Result:

A clear, descriptive error message such as:
"Cannot move item to cart: Item is out of stock."

Actual Result:

A vague error appears:
"Failed to move item to cart."
No reason is provided, leading to poor user clarity.

Impact: Users may retry repeatedly and cant understand the reason for this error , think the site is broken, or contact support unnecessary.



Bug 07 — Product Images Have Inconsistent Aspect Ratios (UI/UX Layout Issue)

Severity: Medium
Module: Product Catalog / Most Popular Section

Description:

Product images across the catalog do not follow a uniform aspect ratio. Some are square, some rectangular, leading to an uneven product grid and visual imbalance.

Steps to Reproduce:

1. Navigate to the **Most Popular** or **Catalog** page.
2. Compare product cards such as:
    - **"Retro Colorblock"**
    - **"Vibrant Pulse"**
3. Observe the image dimensions and layout alignment.

Expected Result:

All product images should maintain a **consistent aspect ratio** (e.g., 1:1 square) to keep the grid aligned and visually uniform.

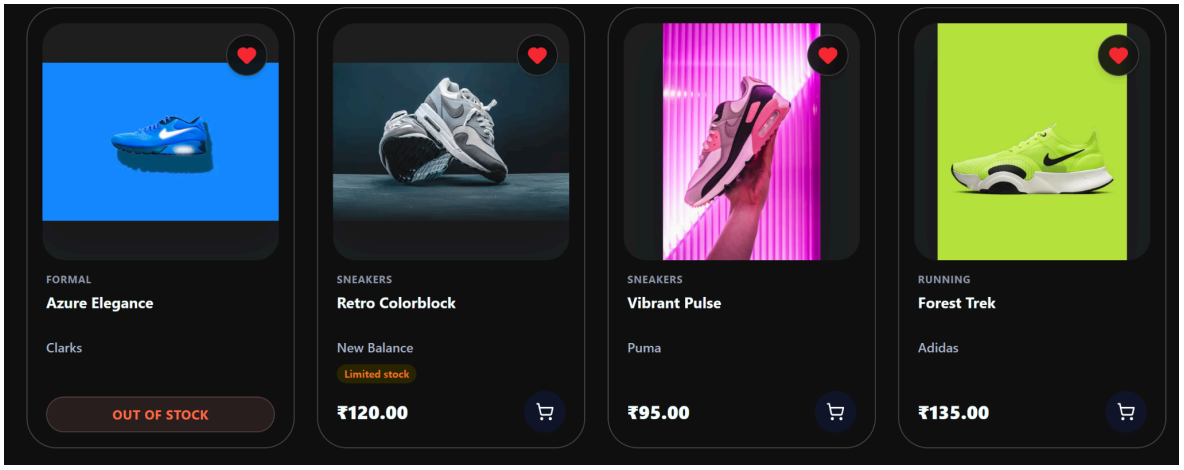
Actual Result:

Images appear in **different orientations** (horizontal, vertical), causing:

- Misalignment of product cards
- Uneven spacing
- A less professional UI

Impact:

Reduces visual consistency and damages perceived product quality and brand professionalism.



3. CROSS-PLATFORM TESTING RESULTS

Platform	Status	Issues
Chrome (Desktop)	Stable	Toast stack bug, form validation bugs
Firefox (Desktop)	Stable	Alignment slightly shifted
Edge (Desktop)	Stable	Product image grid mismatch
iPhone 12 Mobile View	Working	“Shop Now” correctly centered
iPhone SE View	Minor issues	Buttons slightly compressed
Samsung Galaxy S9 View	Good	Product images inconsistent

4. AUTOMATION TEST CASES

AUT_01 – Mandatory field validation

AUT_02 – Numeric name rejection

AUT_03 – Invalid email format check

AUT_04 – PIN code length validation

AUT_05 – Payment method selection behavior

TC-17	Form Validation Automation	Verify mandatory fields block submission when empty.	User is on the Checkout page.	1	Click "Place Order" with all fields empty.	System should display validation errors for required fields.	Form remained on page; validation triggers appeared	**Passed**
TC-18	Data Integrity Automation	Verify the Name field does not accept numeric characters.	User is on the Checkout page.	1	Enter 555555555 into the Full Name field.	System should show an error: "Name cannot contain numbers."	Field accepted numeric input without error.	**Failed**
TC-19	Email Validation Automation	Verify the system detects invalid email formats.	User is on the Checkout page.	1	Enter areesha.com into the Email field.	System should show an error: "Invalid email format."	Red error message "Invalid email" appeared.	**Passed**
TC-20	Data Logic Automation	Verify the PIN Code field enforces a standard length.	User is on the Checkout page.	1	Enter 123 into the PIN code field.	System should require a minimum of 6 digits.	Field accepted a 3-digit PIN.	**Failed**
TC-21	Selection Logic Automation	Verify payment method selection updates correctly.	User has filled contact details.	1	Click the "Cash on Delivery" radio/button	"Cash on Delivery" should be highlighted/selected.	Button selected successfully as confirmed by script.	**Passed**

I used Selenium and Python for automating these 5 Testcases :
Here is the script :

File Edit Selection View Go Run Terminal Help selenium

driver_setup.py

utils > driver_setup.py > ...

```python
def run_aurazone_stable_submission():


    name_xpath = "//input[@placeholder='Enter your full name']"
    name_field = wait.until(EC.element_to_be_clickable((By.XPATH, name_xpath)))
    slow_print("Checkout Form is ready. Beginning 5-Case Suite...")

    # --- TEST CASES START ---

    # AUT_01: Mandatory Check
    place_order_btn = driver.find_element(By.XPATH, "//button[contains(text(), 'Place Order')]")
    driver.execute_script("arguments[0].scrollIntoView({block: 'center'});", place_order_btn)
    time.sleep(1)
    place_order_btn.click()
    slow_print("AUT_01: Clicked Place Order (Empty Check).")
    time.sleep(2)

    # AUT_02: Numeric Name
    name_field.clear()
    name_field.send_keys("555555555")
    slow_print("AUT_02: Entered numeric name.")
    time.sleep(1)

    # AUT_03: Email
    email_field = driver.find_element(By.XPATH, "//input[@placeholder='you@example.com']")
    email_field.send_keys("areesha.com")
    slow_print("AUT_03: Entered invalid email.")
    time.sleep(1)


    fields = {
        "//input[@placeholder='98765 43210']": "03001234567",
        "//input[@placeholder='e.g., 123 Main Street']": "NED University Road",
        "//input[@placeholder='e.g., Mumbai']": "Karachi",
        "//input[@placeholder='e.g., Maharashtra']": "Sindh",
        "//input[@placeholder='e.g., 400001']": "123" # AUT_04
    }

    for xpath, value in fields.items():
        element = driver.find_element(By.XPATH, xpath)
        element.send_keys(value)
        time.sleep(0.5)

    slow_print("AUT_04: Address and short PIN entered.")

    # AUT_05: Payment
    cod_btn = wait.until(EC.element_to_be_clickable((By.XPATH, "//button[contains(., 'Cash on Delivery')]")))
    cod_btn.click()
    slow_print("AUT_05: Cash on Delivery selected.")

    time.sleep(2)
    driver.save_screenshot("final_stable_results.png")
    slow_print("--- ALL TESTS FINISHED SUCCESSFULLY ---")
```

File Edit Selection View Go Run Terminal Help selenium

driver_setup.py

utils > driver_setup.py > ...

```python
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import time

driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()))
driver.implicitly_wait(5)
wait = WebDriverWait(driver, 30)

def slow_print(text):
    """Helper to make the console logs feel more professional"""
    print(f"[LOG]: {text}")
    time.sleep(1.5)

def run_aurazone_stable_submission():
    try:
        # ---  LOAD PRODUCTS ---
        driver.get("https://test.aurazone.shop/products")
        driver.maximize_window()
        slow_print("Page opened. Waiting for products to load...")

        # ---  ADD TO CART ---
        add_btn = wait.until(EC.element_to_be_clickable((By.XPATH, "//button[@aria-label='Add to cart']")))
        # Scroll to button just in case
        driver.execute_script("arguments[0].scrollIntoView({block: 'center'});", add_btn)
        time.sleep(1) # Visual pause
        add_btn.click()
        slow_print("TC_01: Product added successfully.")

        # --- OPEN CART ---
        cart_icon = wait.until(EC.element_to_be_clickable((By.XPATH, "//button[@aria-label='Shopping cart']")))
        cart_icon.click()
        slow_print("TC_02: Shopping cart opened. Reviewing items...")
        time.sleep(2) # Give the side-drawer time to animate open

        # --- PROCEED TO CHECKOUT ---
        proceed_xpath = "//button[contains(text(), 'Proceed to Checkout')]"
        proceed_btn = wait.until(EC.element_to_be_clickable((By.XPATH, proceed_xpath)))
        proceed_btn.click()
        slow_print("TC_03: Proceed button clicked. Moving to Checkout...")


        time.sleep(4)
        if "checkout" not in driver.current_url:
            slow_print("Detected slow redirect. Forcing URL...")
            driver.get("https://test.aurazone.shop/checkout")


        name_xpath = "//input[@placeholder='Enter your full name']"
        name_field = wait.until(EC.element_to_be_clickable((By.XPATH, name_xpath)))
        slow_print("Checkout Form is ready. Beginning 5-Case Suite...")
```

```python
        time.sleep(2)
        driver.save_screenshot("final_stable_results.png")
        slow_print("--- ALL TESTS FINISHED SUCCESSFULLY ---")

    except Exception as e:
        print(f"!!! CRITICAL ERROR: {e}")
        driver.save_screenshot("crash_report.png")
    finally:
        time.sleep(3)
        driver.quit()

if __name__ == "__main__":
    run_aurazone_stable_submission()
```

This Selenium script automates the end-to-end checkout flow on the **AuraZone Products, Cart, and Checkout pages**, validating five critical test cases.

1. Products Page

- The script opens:
  https://test.aurazone.shop/products
- It clicks **"Add to Cart"** on the first available product.

2. Shopping Cart Page

- Opens the cart using the **shopping cart icon**.
- Clicks **"Proceed to Checkout"** to continue.

3. Checkout Page

Once on the Checkout page, the script performs all five automated test cases:

AUT_01 – Mandatory Field Validation

- Clicks **"Place Order"** with **all fields empty**
  → Ensures the system shows required-field validation messages.

AUT_02 – Invalid Name Check

- Enters **"555555555"** into the **Full Name** field
  → Tests incorrect acceptance of numeric-only names.

AUT_03 – Invalid Email Format

- Enters **"areesha.com"** into the **Email** field
  → Verifies that invalid email formats trigger an error.

AUT_04 – PIN Code Length Validation

- Enters **"123"** into the **PIN Code** field
  → Tests whether short PIN values are incorrectly accepted.

Required Fields Filled for Flow Continuity

The script inputs valid data for: Phone number ,Street address , City , State

This ensures the form can progress to the payment validation step.

AUT_05 – Payment Method Selection

- Clicks **"Cash on Delivery"**
  → Confirms the payment option is selectable and updates correctly.

File Edit Selection View Go Run Terminal Help

driver_setup.py

utils > driver_setup.py > run_aurazone_final_submission

```
1  from selenium import webdriver
2  from selenium.webdriver.chrome.service im
3  from webdriver_manager.chrome import Chro
4  from selenium.webdriver.common.by import
5  from selenium.webdriver.support.ui import
6  from selenium.webdriver.support import ex
```

PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL POR

```
dled\x5clibs\x5cdebugpy\x5clauncher' '52695' '--'
PS C:\Users\user\Downloads\selenium>  c:; cd 'c:\
s\ms-python.debugpy-2025.18.0-win32-x64\bundled\l
TC_01: Product added successfully.
TC_02: Shopping cart opened.
TC_03: Proceeded to Checkout.
Waiting for Checkout Form to become visible...
AUT_01: Mandatory field check triggered.
AUT_02: Numeric name entered.
AUT_03: Invalid email entered.
AUT_04: Short PIN entered.
AUT_05: Cash on Delivery selected.
--- FULL TEST SUITE COMPLETE ---
PS C:\Users\user\Downloads\selenium>
```

5. QA Thinking – High-Risk Areas & What I Would Prioritize with More Time

During analysis and exploratory testing, several high-risk areas became evident. These areas either directly affect the user's ability to complete core flows (checkout, add to cart, wishlist) or significantly impact credibility, trust, and user experience. If provided additional time, I would prioritize deeper exploration and more extensive reporting in the following domains:

1. UI/UX Consistency and Layout Stability (High Risk)

The application has noticeable inconsistencies in layout and styling, especially in product listings, filtering results, and image aspect ratios. For example:

- Filtered product results sometimes appear **in smaller or misaligned text**, reducing readability.
- Product images follow **inconsistent aspect ratios**, breaking the grid layout and weakening visual coherence.

Risk Impact:
Poor UI consistency can reduce user trust, affect conversion rates, and create a negative brand impression.

If given more time, I would:

- Report additional layout inconsistencies across different screen sizes.
- Validate typography, spacing, and alignment against a UI style guide.
- Document cross-platform visual deviations in detail.

2. Email Formatting & Communication Clarity (Medium–High Risk)

Order confirmation emails include mixed formatting for address details (PIN, city, address line). This can lead to confusion and reduces the professional quality of transactional communication.

Risk Impact:
Improperly formatted emails can lead to delivery/address issues and create customer confusion post-order.

If given more time, I would:

- Analyze all email templates (order confirmation, cancellation, shipping).
- Validate field placement and perform data-driven tests with edge cases (long city names, special characters).
- Provide revised UX recommendations for clean and readable email formatting.

3. Missing User-Friendly Inventory Controls (Medium Risk)

For Out-of-Stock items, users currently see only an "OUT OF STOCK" button, with **no option to be notified when stock returns**. This impacts user retention and re-engagement.

Risk Impact:
Lack of a "Notify Me" feature reduces potential conversions and can frustrate users wanting specific items.

If given more time, I would:

- Propose and validate "Notify Me" workflows.
- Test edge cases (email format, OTP, multiple notifications).
- Assess integration with backend inventory update events.

4. Validation Logic & Error Messaging (High Risk)

Checkout validation still accepts:

- Numeric names
- Short PIN codes
- Incorrect or ambiguous error messages (e.g., vague “Failed to move item to cart”)

These issues can directly affect the order completion flow.

Risk Impact:
Incorrect validation leads to corrupted customer data and reduces reliability of orders, shipping, and analytics.

If given more time, I would:

- Test all validation boundaries thoroughly (min/max lengths, special characters).
- Create negative test cases for every form input.
- Report error-message clarity and consistency issues in detail.

5. Cross-Platform Behavior Differences (Medium Risk)

Some UI elements (buttons, text, spacing) behave differently in mobile vs. desktop views. These inconsistencies may become severe on smaller screens.

If given more time, I would:

- Expand device matrix testing (iOS, smaller Android screens, tablets).
- Verify orientation changes (portrait → landscape).
- Capture layout breaks and responsiveness issues.

6. Recommendations

Based on the identified issues and overall assessment of the application, the following improvements are recommended:

1. Strengthen Form Validation for Better Security

Several fields—such as *Full Name* and *PIN Code*—accept invalid or incomplete input. Implementing stricter client-side and server-side validation will:

- Reduce invalid order submissions
- Improve data quality
- Prevent potential misuse or malformed entries

2. Improve UI/UX Consistency

Some UI components (product image aspect ratios, stacked notifications, filter text formatting) create a non-professional or cluttered user experience. Enhancing UI consistency would include:

- Using uniform aspect ratios for product cards
- Preventing repeated toast notifications from stacking
- Ensuring filter labels and UI text remain readable and properly aligned

3. Add Missing User-Centric Features (e.g., “Notify Me”)

Out-of-stock product cards currently hide the product price and do not provide an alert option. Adding a **“Notify Me When Available”** button will:

- Improve customer engagement
- Reduce lost sales
- Help users track restock items easily

4. Enhance Wishlist & Inventory Logic

A wishlist showing **“In Stock”** for an item that is actually out of stock can mislead the user. Ensuring real-time synchronization between catalog inventory and wishlist state will improve trust and reduce confusion.